



S I S T E M A S
D I S T R I B U I D O S

Práctica 5.

Objetos distribuidos.

M I C H E L L E A N G U I A N O R O D E A

E S C O M

P R O F E S O R C H A D W I C K C A R R E T O
A R E L L A N O

G R U P O : 7 C M 6



Instituto
politécnico
nacional

E S C O M

1. Antecedentes (Introducción)

La evolución de los sistemas distribuidos ha permitido que distintos programas, incluso en diferentes máquinas y lenguajes de programación, puedan comunicarse y cooperar entre sí.

Uno de los enfoques más importantes en esta área es el de los objetos distribuidos, que permite que un objeto ubicado en una máquina (servidor) sea utilizado por otro programa remoto (cliente) como si se encontrara en el mismo entorno local.

Estos objetos se comunican a través de protocolos de red (TCP/IP) y mecanismos de invocación remota, donde la comunicación es transparente al programador. El cliente puede invocar métodos remotos como si fueran locales gracias a un intermediario (middleware).

Entre las principales tecnologías que implementan este paradigma se encuentran:

- **CORBA (Common Object Request Broker Architecture)**
Permite la comunicación entre objetos escritos en distintos lenguajes de programación (C++, Java, Python, etc.). Utiliza un **ORB (Object Request Broker)** como intermediario para enviar peticiones y respuestas entre clientes y servidores distribuidos.
- **RMI (Remote Method Invocation)**
Es el modelo nativo de **Java** para objetos distribuidos. Permite invocar métodos de objetos ubicados en otras máquinas virtuales Java (JVM). Es una evolución de los sockets pero orientada a objetos.
- **COM/DCOM (.NET)**
Es la versión de **Microsoft** del modelo de objetos distribuidos. COM (Component Object Model) permite la comunicación local entre objetos, y **DCOM (Distributed COM)** extiende esta comunicación a entornos de red, siendo la base de los servicios distribuidos de Windows y .NET.

Estas tecnologías comparten la misma idea central: invocar métodos remotos de manera transparente, permitiendo que la complejidad de la red quede oculta al desarrollador.

2. Problemática

El reto de esta práctica fue comprender y aplicar los fundamentos de los objetos distribuidos, analizando las diferencias entre CORBA, RMI y COM/DCOM, y cómo cada tecnología implementa la comunicación entre procesos remotos.

Los objetivos principales fueron:

1. Entender cómo un cliente puede invocar métodos remotos en un servidor a través de la red.
2. Implementar un ejemplo funcional con RMI en Java para demostrar la comunicación entre objetos distribuidos.
3. Comprender el papel del middleware en la conexión cliente-servidor y cómo se gestiona la serialización de objetos.
4. Comparar las ventajas y limitaciones de CORBA, RMI y COM/DCOM en términos de compatibilidad, lenguajes y plataformas.

3. Solución

Para demostrar el funcionamiento de los objetos distribuidos, se desarrolló un ejemplo práctico con Java RMI.

El sistema consta de tres partes principales:

- Interfaz remota (Hello.java)
Define los métodos que podrán ser invocados remotamente por los clientes.
- Servidor (HelloServer.java)
Implementa la interfaz remota y la registra en el *RMI Registry*, quedando disponible para los clientes.
- Cliente (HelloClient.java)
Localiza el objeto remoto a través del *RMI Registry* y ejecuta métodos como si fueran locales.

El flujo de comunicación es el siguiente:

1. El servidor publica el objeto remoto.
2. El cliente obtiene una referencia al objeto remoto usando `Naming.lookup()`.
3. El cliente invoca un método remoto, y el ORB (Object Request Broker de RMI) se encarga de transmitir la petición y la respuesta.

4. Implementación

Código base en Java RMI.

Hello. Java

```
Hello.java > Hello
1  import java.rmi.Remote;
2  import java.rmi.RemoteException;
3
4  // Interfaz remota
5  public interface Hello extends Remote {
6      String sayHello(String name) throws RemoteException;
7  }
```

HelloServer.java

```
HelloServer.java > HelloServer > main(String[])
1  import java.rmi.registry.LocateRegistry;
2  import java.rmi.registry.Registry;
3  import java.rmi.server.UnicastRemoteObject;
4
5  public class HelloServer implements Hello {
6
7      public String sayHello(String name) {
8          return "Hola " + name + ", saludo desde el servidor RMI.";
9      }
10
11      Run | Debug
12      public static void main(String[] args) {
13          try {
14              HelloServer obj = new HelloServer();
15              Hello stub = (Hello) UnicastRemoteObject.exportObject(obj, port:0);
16
17              // Crear y registrar el objeto remoto
18              Registry registry = LocateRegistry.createRegistry(port:1099);
19              registry.bind(name:"Hello", stub);
20
21              System.out.println(x:"Servidor RMI listo y esperando conexiones...");
22          } catch (Exception e) {
23              System.err.println("Error en el servidor: " + e.getMessage());
24          }
25      }
26  }
```

HelloClient.java

```
HelloClient.java > HelloClient
1  import java.rmi.registry.LocateRegistry;
2  import java.rmi.registry.Registry;
3
4  public class HelloClient {
5      public static void main(String[] args) {
6          try {
7              Registry registry = LocateRegistry.getRegistry(host:"localhost");
8              Hello stub = (Hello) registry.lookup(name:"Hello");
9
10             String response = stub.sayHello(name:"Michelle");
11             System.out.println("Respuesta del servidor: " + response);
12         } catch (Exception e) {
13             System.err.println("Error en el cliente: " + e.getMessage());
14         }
15     }
16 }
17
```

5. Resultados

Durante la ejecución se observó que:

- El cliente pudo conectarse exitosamente al servidor mediante el registro RMI.
- Los métodos definidos en la interfaz remota se invocaron correctamente, demostrando la comunicación cliente-servidor distribuida.
- La invocación remota se comportó como una llamada local, ocultando la complejidad del transporte TCP/IP.

Además, se realizó una comparación conceptual entre las tecnologías:

Tecnología	Lenguaje principal	Plataforma	Middlewa re	Comunicaci ón	Ventaja principal
CORBA	Multilengua je (C++, Java, Python)	Multiplatafor ma	ORB (Object Request Broker)	TCP/IP, IIOP	Interoperabilid ad entre lenguajes
RMI	Java	Multiplatafor ma (JVM)	RMI Registry	TCP/IP	Integración nativa en Java

COM/DCOM	C#, C++, VB	Windows / .NET	DCOM Runtime	RPC sobre TCP	Integración con sistema operativo Windows
-----------------	-------------	-------------------	-----------------	------------------	--

```
PS C:\Users\miaro\Documents\Sistemas distribuidos\Practica 5> & 'C:\Program Files\Java\jdk-25\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\miaro\AppData\Roaming\Code\User\workspaceStorage\6c3fc76a96d8c3eafb0a11ff473983f4\redhat.java\jdt_ws\Practica 5_c2637520\bin' 'Hello Server'
Servidor RMI listo y esperando conexiones...
```

```
PS C:\Users\miaro\Documents\Sistemas distribuidos\Practica 5> & 'C:\Program Files\Java\jdk-25\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\miaro\AppData\Roaming\Code\User\workspaceStorage\6c3fc76a96d8c3eafb0a11ff473983f4\redhat.java\jdt_ws\Practica 5_c2637520\bin' 'Hello Client'
Respuesta del servidor: Hola Michelle, saludo desde el servidor RMI.
PS C:\Users\miaro\Documents\Sistemas distribuidos\Practica 5>
```

7. Referencias

- Oracle. (n.d.). *Java RMI Overview*.
<https://docs.oracle.com/javase/8/docs/technotes/guides/rmi/>
- OMG. (n.d.). *Common Object Request Broker Architecture (CORBA)*.
<https://www.omg.org/spec/CORBA/>
- Microsoft Docs. (n.d.). *Component Object Model (COM)*.
<https://learn.microsoft.com/en-us/windows/win32/com/component-object-model-com--portal>